

Unix & Shell Programming

Module 1 – Introduction to UNIX

Question & Answer Notes

BCA Semester 6 · MAKAUT

Q1. What is UNIX? Explain its key features.

UNIX is a powerful, multi-user, multitasking operating system originally developed in 1969 at Bell Labs by Ken Thompson, Dennis Ritchie, and others. It was written in C language, making it portable across different hardware platforms.

Key Features of UNIX:

1. **Multi-user:** Multiple users can use the system simultaneously. Each user has their own account and permissions.
2. **Multitasking:** UNIX can run many processes at the same time. For example, you can download a file while editing a document.
3. **Portability:** Since UNIX is written in C, it can be moved (ported) to different hardware with minimal changes.
4. **Security:** UNIX provides file permission system (read, write, execute) for owner, group, and others. This keeps data safe.
5. **Hierarchical File System:** Files are organized in a tree structure starting from root (/).
6. **Shell Interface:** UNIX provides a command-line interface called *shell* to interact with the OS.
7. **Pipes and Redirection:** Output of one command can be given as input to another using pipes (`|`), and I/O can be redirected.
8. **Open Source Friendly:** The UNIX design inspired many open-source operating systems like Linux.
9. **Networking Support:** UNIX was one of the first OS to support TCP/IP networking.
10. **Everything is a File:** In UNIX, devices, directories, and processes are treated as files, making it consistent and simple.

Remember: UNIX follows the philosophy – “*Do one thing and do it well.*” Each command does a specific job.

Q2. What is POSIX? Why is it important?

POSIX stands for **P**ortable **O**perating **S**ystem **I**nterface. It is a set of standards defined by IEEE (Institute of Electrical and Electronics Engineers) to maintain compatibility between different operating systems.

In simple words, POSIX is a set of rules that an OS must follow so that programs written for one UNIX-like system can run on another without major changes.

Why is POSIX Important?

1. **Portability:** A program written on Linux can run on macOS or other UNIX systems if both follow POSIX standards.
2. **Standardization:** POSIX defines standard system calls, library functions, shell commands, and utilities (like `ls`, `cp`, `grep`).
3. **Consistency:** Developers don't need to rewrite programs for every OS. One code works on all POSIX-compliant systems.
4. **Wide Adoption:** Linux, macOS, Solaris, BSD – all follow POSIX standards.
5. **Easier Development:** Application developers can target POSIX and be confident their software runs on multiple platforms.

Examples of POSIX-compliant systems: Linux, macOS, FreeBSD, Solaris, AIX.

Short Answer: POSIX = Rules that make programs work across different UNIX-like systems without change.

Q3. Discuss Linux and list the most popular Linux distributions.

Linux is a free, open-source operating system kernel created by **Linus Torvalds** in 1991. It was inspired by UNIX but written from scratch. Linux is the kernel – the core part of the OS that manages hardware resources.

Key Points about Linux:

- Free and open source (source code is available to everyone).
- Used in servers, desktops, smartphones (Android), embedded systems, supercomputers.
- Very stable, secure, and fast.
- Supported by a large community worldwide.
- Licensed under GPL (GNU General Public License).

Popular Linux Distributions (Distros):

A *distribution* is a complete OS built using the Linux kernel + additional software.

Distribution	Description
Ubuntu	Most popular for beginners. User-friendly, good community support.
Fedora	Cutting-edge features, sponsored by Red Hat.
Debian	Very stable and reliable. Base for Ubuntu.
CentOS / RHEL	Used in enterprise servers. RHEL = Red Hat Enterprise Linux.
Kali Linux	Used for ethical hacking and security testing.
Arch Linux	For advanced users who want full control.
Linux Mint	Beginner-friendly, looks similar to Windows.
Android	Linux-based OS for mobile devices.

Q4. Compare UNIX and Linux.

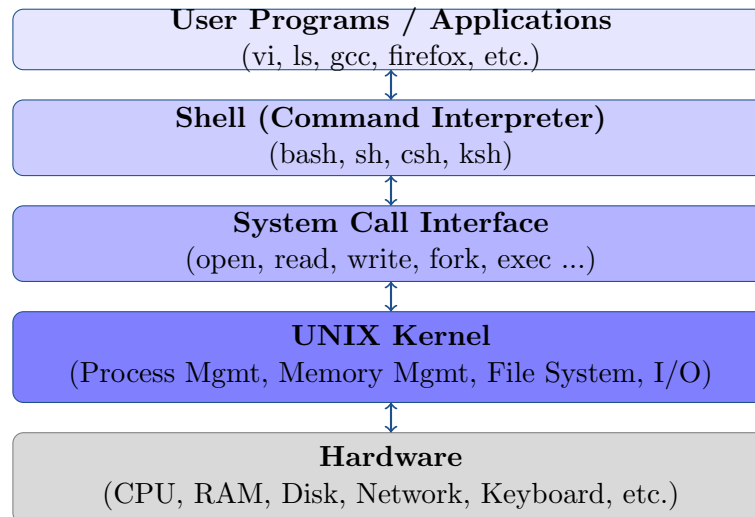
Both UNIX and Linux are similar in design and behavior, but they have important differences:

Feature	UNIX	Linux
Origin	Developed at Bell Labs, 1969	Created by Linus Torvalds, 1991
Cost	Mostly paid/proprietary	Free and open source
Source Code	Closed (mostly)	Open source (available to all)
Portability	Limited to specific hardware	Runs on almost any hardware
Usage	Servers, workstations	Servers, desktops, mobile, IoT
Examples	Solaris, AIX, HP-UX	Ubuntu, Fedora, Debian
File System	UFS, JFS	ext3, ext4, XFS, Btrfs
Community	Commercial support	Huge open-source community
POSIX	Fully compliant	Mostly compliant

Simple way to remember: Linux is *inspired by* UNIX but is not UNIX. Linux is free; most UNIX versions are paid.

Q5. Explain the UNIX system architecture with a neat diagram.

The UNIX system is organized in layers. Each layer communicates only with the layer next to it.



Explanation of each layer:

1. **Hardware:** The physical components – CPU, memory, disk drives, keyboard, monitor.
2. **Kernel:** The core of UNIX. It directly controls hardware. It handles:
 - Process management (creating, scheduling processes)
 - Memory management (allocating/freeing RAM)
 - File system management
 - Device I/O control
3. **System Call Interface:** The bridge between user programs and the kernel. User programs request kernel services through system calls like `open()`, `read()`, `write()`, `fork()`.
4. **Shell:** A program that reads user commands and executes them. It acts as the interface between the user and the kernel.
5. **User Programs / Applications:** Programs that users run, like text editors, compilers, web browsers.

Q6. What is the UNIX kernel? Explain the system call interface.

The UNIX Kernel:

The **kernel** is the heart (core) of the UNIX operating system. It is the first program loaded when the OS boots, and it stays in memory all the time.

The kernel manages:

- **Process Management** – Creates, schedules, and terminates processes.

- **Memory Management** – Allocates and deallocates memory to programs.
- **File System** – Manages files and directories on disk.
- **I/O Management** – Controls communication with devices (keyboard, disk, printer).
- **Security and Access Control** – Enforces permissions and user rights.
- **Network Communication** – Manages network protocols.

The kernel runs in **privileged mode** (also called kernel mode), where it can access all hardware directly.

System Call Interface:

User programs run in **user mode** and cannot directly access hardware. When a program needs to do something like read a file or create a process, it requests the kernel using a **system call**.

A **system call** is a function provided by the kernel that user programs can call to request a service.

How it works:

Step 1: User program calls a library function (e.g., `printf()`).

Step 2: The library function triggers a system call (e.g., `write()`).

Step 3: The CPU switches from user mode to kernel mode.

Step 4: The kernel performs the requested operation.

Step 5: Result is returned back to the user program.

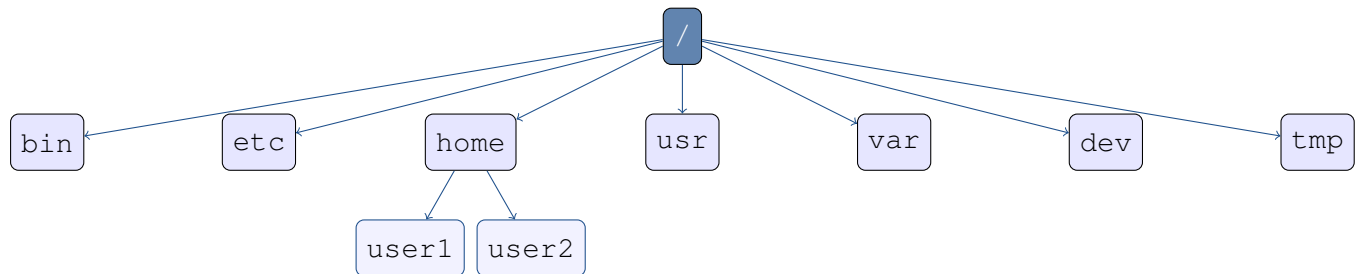
Common System Calls:

Category	System Call	Purpose
File	<code>open</code> , <code>read</code> , <code>write</code> , <code>close</code>	Work with files
Process	<code>fork</code> , <code>exec</code> , <code>exit</code> , <code>wait</code>	Create/manage processes
Directory	<code>mkdir</code> , <code>rmdir</code> , <code>chdir</code>	Manage directories
Info	<code>getpid</code> , <code>getuid</code> , <code>time</code>	Get system information
Communication	<code>pipe</code> , <code>socket</code>	Process/network communication

Analogy: Kernel is like a manager of a company. Users (employees) don't directly touch the machines – they request the manager (kernel) to do things via system calls.

Q7. Explain the UNIX directory structure.

UNIX uses a **hierarchical file system** – files and directories are organized in a tree shape. Everything starts from a single root directory / (called *root*).



Important Directories and Their Purpose:

Directory	Description
/	Root directory. The top of the file system tree. All other directories are inside this.
/bin	Contains essential binary (executable) commands needed to boot the system. Example: ls, cp, mv, cat.
/etc	Contains system configuration files. Example: /etc/passwd (user info), /etc/hosts.
/home	Contains home directories of normal users. Example: /home/shadow for user shadow.
/root	Home directory of the root (superuser/admin).
/usr	Contains user-installed programs, documentation, and libraries.
/var	Contains variable data that changes often – logs, mail, print spool.
/tmp	Temporary files. Cleared on reboot.
/dev	Contains device files. Example: /dev/sda (hard disk), /dev/tty (terminal).
/lib	Contains shared library files needed by programs in /bin and /sbin.
/sbin	Contains system administration commands (only for root). Example: fdisk, ifconfig.
/proc	Virtual directory – contains information about running processes and the kernel.
/mnt	Mount point for temporarily mounted file systems (USB, CD-ROM).

Important Terms:

- **Absolute Path:** Full path from root. Example: `/home/shadow/notes.txt`
- **Relative Path:** Path from current directory. Example: `../documents/file.txt`
- `.` (dot) – refers to the current directory.
- `..` (double dot) – refers to the parent directory.

Key Point: In UNIX, *everything is a file* – even hardware devices and directories are represented as files in the file system.